

tham

ABC460 题解

C - SUSHI

题意

你有 N 个寿司饭团和 M 个配料，作为制作寿司的食材。

第 i 个饭团的重量是 A_i ，第 j 个配料的重量是 B_j 。

你将通过搭配饭团和配料来制作寿司。

要制作一个寿司，你需要将一个饭团和一个配料组合在一起。这里有一个限制条件：**配料的重量不能超过饭团重量的两倍**（即 $B_j \leq 2 \times A_i$ ）。此外，同一个饭团或配料不能被用于多个寿司中（每个只能用一次）。

请计算最多能制作多少个寿司。

Constraints

- $1 \leq N, M \leq 2 \times 10^5$
- $1 \leq A_i, B_j \leq 10^9$



SOLUTION

- 先对所有饭团与所有配料按重量从小到大排序。
- 接下来可以从饭团角度出发选择配料，或者从配料角度出发选择饭团均可。下面以饭团角度进行讨论。
- 对于重量越大的饭团，其对应的配料重量条件就会越宽松，因此接下来先按重量从大到小看一遍所有的饭团。
- 由于能与饭团搭配的配料重量不能超过饭团重量的两倍，因此对于配料而言，重量越大有可能就越不好搭配。所有我们应该尽可能优先地用掉重量更大的配料。
- 所以对于重量为 A_i 的饭团，我们可以优先选择重量不超过 $2A_i$ 的所有配料当中重量最大的配料来与其搭配。
- 而当搭配完重量较大的饭团后，随着剩余饭团重量逐渐变小，能够与其搭配的配料重量也会随之变小，因此可以借助双指针来维护目前还能够选择的配料中重量最大的配料。
- 时间复杂度为排序的复杂度



```
int n, m;
int a[200005], b[200005];

void solve()
{
    cin >> n >> m;
    for(int i = 1; i <= n; i++)
        cin >> a[i];
    for(int i = 1; i <= m; i++)
        cin >> b[i];

    sort(a + 1, a + n + 1);
    sort(b + 1, b + m + 1);

    int ans = 0;

    // 从大到小看一遍所有饭团
    // j 维护此时还没用过的配料中 重量最大的配料
    for(int i = n, j = m; i >= 1; i--)
    {
        // 只要此时重量最大的配料没法用，就往前看重量偏小的
        while(j >= 1 && b[j] > 2 * a[i])
            j--;
        // 如果越界，说明配料已经全部用完
        if(j < 1)
            break;
        else
        {
            ans++;
            j--;
        }
    }

    cout << ans;
}
```

D - REPEATEDLY REPAINTING

题意

有一个 H 行 W 列的网格图。

每个单元格被染成白色或黑色。字符 `.` 表示白色，字符 `#` 表示黑色。

你将执行以下操作 10^{100} 次：

- 每次操作，对所有单元格同时应用以下规则：
 - 在操作前为白色的单元格，当且仅当它周围存在至少一个相邻的黑色单元格时，会变成黑色。这里的“相邻”是指两个单元格 (x, y) 和 (x', y') 八相邻（即 $\max(|x - x'|, |y - y'|) = 1$ ，包含上下左右及四个对角线方向）。
 - 在操作前为黑色的单元格，会变成白色。

输出所有操作结束后，每个单元格的最终颜色。

数据范围

- $1 \leq H \times W \leq 10^6$
- H 和 W 为正整数。
- S_i 为长度为 W 的字符串，由 `.` 和 `#` 构成。



SOLUTION

- 由于 10^{100} 太大了，需要找找在变换操作过程中的规律。
- 观察样例 1 的图：在执行第一次操作后，从黑点能走 1 格到达的白点都被涂成了黑色，黑点变为白色；执行第二次操作后，原图（第一次操作前的图）中的黑点不变，右下角的白点变为了黑点，因为从原图中的黑点到达该点最少需要走 2 步。
由于 10^{100} 是偶数，依此类推，如果一个点到原图中黑点的最短距离为偶数，在偶数次操作后会变为黑点，否则会变为白点。
- 既发现存在一对相邻的格子是一黑一白，很明显接下来每次操作，白的会变黑、黑的会变白，再进行一次操作后又会变回来。也就是说，这样的两个相邻格子在接下来的操作中颜色会相互交替。
- 因此，对于这种格子，只需要关注每个格子第一次变为黑色的时间的奇偶性，其后根据奇偶性来判断在第 10^{100} 次操作完成后是黑格子还是白格子即可。每个格子第一次变为黑色格子的时间，这和每个格子移动到最近的黑色点上的步数是相同的。因此这一部分可以直接借助多源点的宽度优先搜索来实现。
- 但还存在一种特殊情况，如样例二，当有某个黑色点的周围也全是黑色点时，此时该点在 2 次操作后并不能重新变回黑色点。但由于在操作过程中，不可能出现将某个 $3*3$ 的格子全部变为黑色点的情况，因此这种情况只可能出现在地图刚输入的时候。
- 解决方法也很简单，可以先手动模拟一遍第一次操作，去除这种情况，然后再进行宽搜求解。

```

using namespace std;
const int dx[8]={1,0,-1,0,1,1,-1,-1};
const int dy[8]={0,1,0,-1,1,-1,1,-1};
int h,w;
struct node{int x,y;};
queue<node> q;
int main()
{
    scanf("%d%d",&h,&w);
    char a[h+5][w+5],b[h+5][w+5];
    int dis[h+5][w+5];
    for(int i=1;i<=h;i++) scanf("%s",a[i]+1);

    for(int i=1;i<=h;i++)
    {
        for(int j=1;j<=w;j++)
        {
            dis[i][j]=1e9;
            if(a[i][j]=='#')           // 先模拟第一遍，处理掉3*3格子全是黑点的情况
            {                         // 将第一遍，处理得到新图存放到 b[][]
                for(int k=0;k<8;k++)
                {
                    int nx=i+dx[k];
                    int ny=j+dy[k];
                    if(nx<1||nx>h||ny<1||ny>w) continue;
                    if(a[nx][ny]=='.') b[nx][ny]='#';
                    else b[nx][ny]='.';
                }
            }
        }
    }
}

```

```

for(int i=1;i<=h;i++)           // 将黑点的位置放入多源 bfs 起点
{
    for(int j=1;j<=w;j++)
    {
        if(b[i][j]=='#')
        {
            q.push({i,j});
            dis[i][j]=0;
        }
    }
}
while(!q.empty())
{
    int x=q.front().x;
    int y=q.front().y;
    q.pop();
    for(int i=0;i<8;i++)
    {
        int nx=x+dx[i];
        int ny=y+dy[i];
        if(nx<1||nx>h||ny<1||ny>w||dis[nx][ny]!=1e9) continue;
        dis[nx][ny]=dis[x][y]+1;
        q.push({nx,ny});
    }
}
for(int i=1;i<=h;i++)
{
    for(int j=1;j<=w;j++)
    {
        if(dis[i][j]%2==0) printf(".");
        else printf("#");
    }
    printf("\n");
}
return 0;

```

$$E - X + Y \equiv X + Y$$

题意

对于正整数 a 和 b , 定义 $\text{concat}(a, b)$ 为将 a 和 b 前后拼接所形成的整数。例如 , 如果 $a = 123$ 且 $b = 45$, 那么 $\text{concat}(a, b) = 12345$ 。

给定两个正整数 N 和 M 。请找出满足以下条件的正整数对 (x, y) 的数量 (结果对 998244353 取模) :

- x 和 y 均不超过 N
- 满足同余式 $\text{concat}(x, y) \equiv x + y \pmod{M}$ 。

Constraints

- $1 \leq T \leq 10^4$
- $1 \leq N \leq 10^{18}$
- $2 \leq M \leq 10^9$



SOLUTION

考虑到设 l_Y 为 Y 在十进制下的长度，那么题目给的关系式就相当于 $X \times 10^{l_Y} + Y \equiv X + Y \pmod{M}$ 。

容易发现这个式子就可以继续化成 $X \times 10^{l_Y} \equiv X \pmod{M}$ ，由同余的定义知，这就意味着 $M \mid X \times (10^{l_Y} - 1)$ 。

继续转化，设 $g = \gcd(M, 10^{l_Y} - 1)$ ，那么条件又可以变成 $\frac{M}{g} \mid X$ 。

✓ 证明

因为 g 已经把 M 与 $10^{l_Y} - 1$ 的所有公因数提出来了，所以 $\frac{M}{g}$ 这部分因数只能被包含在 X 里。

考虑到只有 l_Y 范围比较小，考虑枚举 l_Y ，那么就可以轻松求出 $\frac{M}{g}$ ，此时 X 的方案数既为 $\lfloor \frac{N}{M/g} \rfloor$ ，最后再乘上 Y 的方案数累加起来就行了。

⚠ 警告

因为 N 最大是 19 位的，所以说求 $10^{l_Y} - 1$ 一定要记得开 unsigned long long，不然可能会爆（我没试过，但应该会卡）。



```

typedef unsignedlonglong ull;
const ull mod = 998244353;

ull gcd(ull a, ull b)
{
    if(b == 0)
        return a;
    return gcd(b, a % b);
}

void solve() // 单组数据 solve
{
    ull n, m;
    cin >> n >> m;

    ull ans = 0;
    ull l = 1, r = 9;
    // 假设满足条件的 y 的长度为 w, 枚举该长度
    // 记 l, r 表示长度为 w 的整数的左右边界
    // l 也可记作 10 的 w-1 次方
    // r 也可记作 10 的 w 次方 - 1
    while(l <= n)
    {
        ull cntY = min(r, n) - l + 1; // n 以内且位数为 w 的正整数个数
        cntY %= mod;

        ull g = gcd(r, m); // 10^w - 1 和 m 的最大公因数
        ull cntX = n / (m / g);
        cntX %= mod;

        ans = (ans + cntX * cntY) % mod;
        l = l * 10;
        r = r * 10 + 9;
    }
    cout << ans << "\n";
}

```

P15451 [JOI 2026 SEMIFINAL] 座位 3 / SEATS 3

○ 题目大意

有 $2N+2$ 个座位排成一行。从左数第 i 个座位 ($1 \leq i \leq 2N+2$) 的舒适度为 A_i 。

现有 N 组结伴而来的两人团体客人，以及 2 名单身来访的 VIP 客人，需要为这 $2N+2$ 位客人每人分配一个座位。但是，不能将同一个座位分配给两位或以上的客人。

现在，对于属于同一团体的两个人，必须分配相邻的座位。在此条件下，我们希望分配给两位 VIP 客人的两个座位的舒适度之和尽可能大。

给定座位的相关信息，请编写一个程序，求出分配给两位 VIP 客人的两个座位的舒适度之和的最大值。

$$1 \leq N \leq 200000$$

$$1 \leq A_i \leq 10^9 \quad (1 \leq i \leq 2N+2)$$

输入的所有值均为整数。



SOLUTION

- 同一团体的两个人必须分配相邻的座位，因此，2名单身来访的 **VIP** 客人的座位，必须一个是奇数号，一个是偶数号。
- 数据范围： $1 \leq N \leq 200000$ ，双重循环暴力枚举两个座位会超时，优化？？
- 运用 **前缀/后缀最大值** 的思想，用一个数组 *si* 来记录在第 *i* 个座位之前奇数号的座位舒适度的最大值，*pi* 记录第 *i* 个座位之后偶数号的座位舒适度的最大值
- 借助预处理的前缀和后缀值，从前往后枚举奇数位置，变量 *ans* 来记录分配给两位 **VIP** 客人的两个座位的舒适度之和的最大值。



P14293 [JOI2024 预选赛 R2] 购物 2 / SHOPPING 2

- 题目大意
 - JOI 商店有 N 件商品，每件商品有一个定价和种类（1 到 M ）。
 - 促销活动持续 M 天，第 j 天时所有种类为 j 的商品按半价出售。
 - 有 Q 位顾客，第 k 位顾客在第 Tk 天购买了编号从 Lk 到 Rk 的所有商品。
 - 要求计算每位顾客购买这些商品的总花费（当天促销的商品按半价，其他按原价）。
-
- 数据范围
 - $1 \leq N, M, Q \leq 2 \times 10^5$ 。
 - $1 \leq Lk \leq Rk \leq N$ ($1 \leq k \leq Q$) 。



- 暴力枚举
- 发现如果不加促销的话，这道题就是道普通前缀和
- 每一次询问，我们可以先求出正常情况下从 L 到 R 中所有商品的价值，然后减去在区间内第 D 天商品的折扣价。
- 前一部分很显然可以通过前缀和完成。
- 然后，我们开一个 **vector** 数组 t ，其中 t_i 存放第 i 天的所有商品的价值与下标。
- 每一次询问中，我们对于 td 二分查找，分别找到第一个下标大于 L 的和最后一个下标小于 R 的商品。将这个区间内的商品价值相加，再除以 2 ，就可以得到这一天在区间内的商品的折扣价。这个相加的过程同样可以前缀和优化。



```

#define int long long
using namespace std;
const int N=2e5+20;
int n,m,q,k;
struct node{
    int id,sum;
};
int x,y,day,l,r;
vector<node> v[N];
int sum[N];
int check(int d,int L,int R){
    int l=0,r=v[d].size()-1,res=-1,res1=-1;
    while(l<=r){
        int mid=l+r>>1;
        if(v[d][mid].id<L) l=mid+1;
        else r=mid-1,res=mid;
    } //找出这段区间内种类为 d 的编号最大的
    l=0,r=v[d].size()-1;
    while(l<=r){
        int mid=l+r>>1;
        if(v[d][mid].id<=R) res1=mid,l=mid+1;
        else r=mid-1;
    } //找出这段区间内种类为 d 的编号最小的
    if(res1==-1) return 0;
    if(v[d][res1].id<L) return 0;
    if(res<=0) return v[d][res1].sum;
    else return v[d][res1].sum-v[d][res-1].sum; //前缀和
}

```

```

signed main()
{
    cin>>n>>m>>q;
    for(int i=1;i<=n;i++){
        cin>>y>>x;
        sum[i]=sum[i-1]+y,k=v[x].size()-1;
        if(k<0) k=y;
        else k=v[x][k].sum+y;
        v[x].push_back((node){i,k}); //这里维护每个种类的前缀和
    }
    for(int i=1;i<=q;i++){
        cin>>day>>l>>r;
        int ans=sum[r]-sum[l-1]-check(day,l,r)/2;
        cout<<ans<<"\n";
    }
}

```



P14294 [JOI2024 预选赛 R2] 白色灯 2 / WHITE LIGHT 2

- 题目大意
- 现有一排初始均亮着的灯，每个灯的颜色都是红、绿、蓝中的其中一个。现在你可以花费 a 的代价关掉最左边的开着的灯，也可以花费 b 的代价关掉最右边的开着的灯，还可以花费 c 的代价改变一个灯的颜色。你需要求出最小的代价，使得所有亮着的灯颜色按红、绿、蓝、红、绿、蓝.....的顺序排列。
- 数据范围
- $1 \leq N \leq 200000$ 。
- S 是长度为 N 的字符串， S 中的每个字符为 R、G、B 中的某一个。

子任务

1. (4分) $N = 3$ 。
2. (22分) $N \leq 300$ 。
3. (19分) $N \leq 10\,000$ 。
4. (9分) N 是 3 的倍数, $A = 10^9$, $B = 10^9$, $C = 1$ 。
5. (10分) $A = 10^9$, $B = 10^9$, $C = 1$ 。
6. (36分) 无额外约束。



- 暴力枚举

- 1. $O(n^3)$ 做法:**

我们发现，最后亮着的灯一定是一个长度为 3 的倍数的连续子区间。所以考虑 $O(n^2)$ 枚举所有长度为 3 的倍数的区间，再暴力判断这个区间内有多少灯需要进行操作 3，才能符合要求。取最小大代价。

- 暴力枚举的优化

- $O(n^2)$ 做法:**

不难发现，中间暴力判断的部分可以进行预处理。

设数组 $S_{i,j}$ 为以第 i 个灯为开头进行循环，到 j 号灯需要操作的次数，第 i 个灯需不需要进行操作三（需要为 1，不需要为 0），这样，对 s 数组进行前缀和，那么 $[l,r]$ 区间内的灯需要进行操作三的次数就是以 l 为开头，到 r 结尾的答案 $S_{l,r}$ 。

预处理和枚举长度为 3 的区间，复杂度都为 $O(n^2)$



○ DP

令 $dp_{i,0/1/2}$ 表示第 i 位为红色/绿色/蓝色 (0/1/2)，且前面均满足红绿蓝重复循环的最小花费。这是不难想的，但是要如何转移呢？

✔ 观察

那么就好做了，对于每个位置有三种情况：

1. 这一位为红色，那么它可以由前一位的蓝色转移而来，当然也可直接将前面的灯全部熄灭，将这一位作为开头。
2. 这一位为绿色，那么它只能由前一位的红色转移而来。
3. 这一位为蓝色，那么它只能由前一位的绿色转移而来。

对于每个位置，如果当前的颜色与原来不同，就要花费 c 的代价变色。

对于每个位置的蓝色，可以作为最终序列的结尾，要把后面的 $n - i$ 位均花费 b 的代价熄灭，那么总共的代价就为 $dp_{i,2} + (n - i) \times b$ 。最后和 $\min(a, b) \times n$ (全部熄灭) 取个最小值，输出即可。

```
const int N=2e5+5;
long long dp[N][3];
char s[N];
int main()
{
    int n;
    // string s;
    long long a,b,c;
    cin>>n>>s+1>>a>>b>>c;
    long long ans=min(a,b)*n;
    memset(dp,0x3f3f3f3f,sizeof dp);

    for(int i=1;i<=n;++i)
    {
        dp[i][0]=min(a*(i-1),dp[i-1][2]);    // 转移的三种情况讨论
        if(s[i]!='R')dp[i][0]+=c;
        dp[i][1]=dp[i-1][0];
        if(s[i]!='G')dp[i][1]+=c;
        dp[i][2]=dp[i-1][1];
        if(s[i]!='B')dp[i][2]+=c;
        ans=min(dp[i][2]+(n-i)*b,ans);
    }
    cout<<ans;
    return 0;
}
```

参考

- <https://github.com/zhongqiangliu/awesome-go>

